

Elasticsearch

1. What is Elasticsearch?

Elasticsearch is a distributed search and analytics engine. It stores JSON documents and makes every field in them searchable.

Property	Detail
Built on	Apache Lucene, which does the actual indexing
Talked to over	HTTP, with JSON request bodies
Distributed	Data is split across nodes and copied, by default
Part of	The Elastic Stack, with Kibana for viewing and Beats for shipping

It is a search engine that happens to store data, not a database that happens to search. That ordering explains most of its behavior.

2. What problem does Elasticsearch solve?

A relational database finds rows whose values match. It cannot answer "which of these documents are most relevant to these words".

Question	A relational database	Elasticsearch
The row with id 41	Fast	Fast
Rows where status is OPEN	Fast, with an index	Fast
Products matching "wireless headphone"	<code>LIKE '%...%'</code> , a full scan, no ranking	Ranked, in milliseconds
Logs mentioning a timeout in the last hour	Slow once there are billions	What it exists for
Group 100 million rows by category	Minutes	Seconds

The difference is relevance and scale, not lookup. A database can find the rows containing a word if it must. It cannot say which of them answers the question best, and it cannot do it over a hundred million rows while somebody waits.

3. Elasticsearch vs a relational database

	Elasticsearch	Relational database
Unit of storage	A JSON document	A row

	Elasticsearch	Relational database
Grouped into	An index	A table
Schema	Inferred, or declared as a mapping	Declared up front
Joins	Effectively none	First class
Transactions	None across documents	Full ACID
Consistency	Near real time	Immediate
Best at	Search, and aggregating at scale	Correctness and relationships

Elasticsearch is normally the second copy, not the source of truth. The database owns the data and Elasticsearch is fed from it, because losing a search index is an inconvenience and losing the ledger is not.

4. Full-text search

Full-text search finds documents containing the words of a query and ranks them by how well they match. It is not an exact comparison.

The work happens when the document is indexed. The text is analyzed: lowercased, split into tokens, reduced to word stems, and common words dropped.

```
"Wireless Bluetooth Headphones" → [wireless, bluetooth, headphon]
```

Those tokens go into an inverted index, which maps each term to the documents containing it:

```
wireless → doc1, doc7
bluetooth → doc1, doc3
headphon → doc1, doc7, doc9
```

A search for "bluetooth headphones" is then a lookup, not a scan, and doc1 ranks first because it contains both terms.

Ranking is the part a database has no answer for. Elasticsearch scores each match on how often the term appears in the document, how rare the term is across all documents, and how long the field is. A rare word matching a short field counts for much more than a common word in a long one.

This is why `LIKE '%headphone%'` **is not the same thing.** It cannot use an index, it does not stem, and every row that matches matches equally.

5. Document-oriented database

The unit of storage is a document: one self-contained JSON object, rather than a row whose related data lives in other tables.

Relational	orders, order_lines, customers, products	joined at query time
Document	one order document containing all of it	joined at write time

There are no joins, so the data is denormalized. A document carries everything a query about it needs, including values copied from what would have been other tables.

The cost is duplication. A product name stored inside ten thousand order documents has to be updated in ten thousand places when it changes, which is work the relational model does once. That trade is accepted because reads are what this is optimized for.

6. JSON documents

A document is JSON, and Elasticsearch wraps it with metadata:

```
{
  "_index": "products",
  "_id": "41",
  "_version": 3,
  "_source": {
    "name": "Wireless Bluetooth Headphones",
    "category": "audio",
    "price": 129.99,
    "inStock": true,
    "tags": ["wireless", "bluetooth"]
  }
}
```

Field	Is
<code>_index</code>	Which index it belongs to
<code>_id</code>	Its identifier, given or generated
<code>_version</code>	Incremented on every write
<code>_source</code>	The original JSON, returned with results

Documents are not modified in place. An update reads the document, applies the change, writes a new version, and marks the old one deleted. The old copy occupies space until a merge removes it, which is why a heavily updated index grows.

7. Near-real-time search

A document is searchable about one second after it is indexed, not immediately.

The reason is how the index is built. New documents go into an in-memory buffer, and a refresh turns that buffer into a searchable segment. The refresh runs once a second by default.

Operation	Sees a new document
<code>GET</code> by id	Immediately
Search	After the next refresh, about a second

PUT /products/_doc/41

then GET /products/_doc/41

found

PUT /products/_doc/41

then POST /products/_search

possibly not yet

This is the behavior that surprises people, and it is worth saying out loud. Writing a document and immediately searching for it can return nothing, while fetching it by id returns it. Nothing is broken.

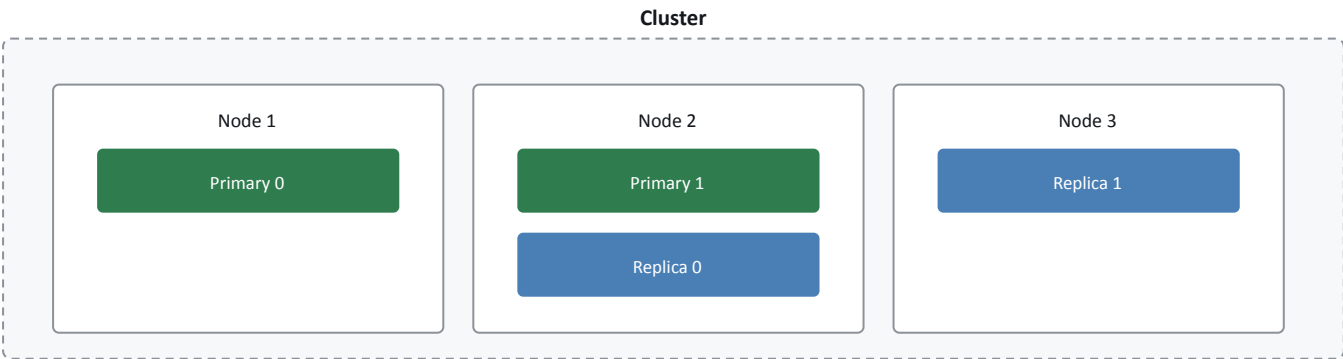
A write can force a refresh, at a real cost:

```
PUT /products/_doc/41?refresh=true
```

Do not use that to work around a test. Refreshing on every write creates a segment per document and destroys indexing throughput. If a test needs it, the test should ask for one refresh at the end.

8. Elasticsearch architecture

The pieces nest: a cluster holds nodes, a node holds shards, and a shard is a Lucene index holding documents.



No node holds both copies of a shard, so losing any one of the three loses no data.

Level	Is
Cluster	Every node working together under one name
Node	One Elasticsearch process
Index	A named set of documents, split into shards
Shard and replica	A Lucene index, and its copy on another node

A search runs on every shard and the results are merged. The node that receives the request sends it to one copy of each shard, collects what comes back, sorts it, and replies. Any node can do this.

9. Cluster

A cluster is one or more nodes sharing a cluster name and holding the data between them.

One node is elected master. It manages cluster state: which indices exist, where the shards are, which nodes are present. It does not sit in the path of searches.

```
GET /_cluster/health
```

Health	Means
Green	Every primary and every replica is assigned
Yellow	Every primary is assigned, some replica is not
Red	At least one primary is missing. Some data cannot be searched

Yellow is normal on a single-node cluster. A replica cannot sit on the same node as its primary, so with one node there is nowhere to put it. That is not a fault and does not need fixing in development.

10. Node

A node is one Elasticsearch process. What it does depends on the roles it is given.

Role	Does
Master-eligible	Can be elected to manage cluster state
Data	Holds shards and runs queries against them
Ingest	Runs preprocessing pipelines before indexing
Coordinating	Routes a request, merges the results

Every node coordinates. Any node can accept a search, fan it out and merge the answers, which is why clients can talk to any of them.

Separate the master role on a large cluster. A master node busy serving queries is a master node that can miss its own election timeouts, and an unstable master is worse than a slow search.

11. Index

An index is a named collection of documents with a similar shape. It is the rough equivalent of a table.

```
PUT /products
{
  "settings": { "number_of_shards": 2, "number_of_replicas": 1 }
}
```

The shard count is fixed when the index is created. Changing it means creating a new index and reindexing into it, which is why an alias is usually put in front from the start: the alias is repointed and no client changes.

Too many small shards is the common mistake. Every shard carries overhead, so a hundred shards holding a megabyte each cost far more than a few holding a gigabyte. A shard in the tens of gigabytes is a reasonable target.

Index names are lowercase, with no spaces, and cannot begin with `-`, `_` or `+`. Beyond that the naming is a design decision:

Pattern	For
<code>products</code>	A stable set of entities
<code>logs-2026.09.16</code>	One index per day
<code>logs-000001</code> behind an alias	Rolled over by size or age

Time-based indices exist so that old data can be dropped, not deleted. Deleting a month of documents is millions of individual marks and a long merge. Deleting the index that holds them is instant and reclaims the space at once.

12. Document

A document is one JSON object in an index, identified by an id.

PUT	/products/_doc/41	create or replace
GET	/products/_doc/41	fetch by id
POST	/products/_update/41	partial update
DELETE	/products/_doc/41	delete

`_id` is the identifier, unique within the index. It is either given or generated, and that choice has consequences:

	A provided id	A generated id
Writing the same document twice	Overwrites. The write is idempotent	Creates a second copy
Indexing speed	Slower. Existence has to be checked	Fastest
Right for	Entities mirrored from a database	Logs and events

Use the database primary key as the **`_id`** when mirroring. Re-running a sync then overwrites rather than duplicating, which makes the whole pipeline safe to repeat.

`_source` is the original JSON, stored verbatim. It is what search results return, and it is what **`update`** and **`reindex`** read to rebuild a document. Disabling it saves space and takes both of those away, which is almost never worth it.

The document lifecycle has five stages, and two of them are confused for each other constantly:

Stage	What happens
Index	Written to an in-memory buffer and to the translog
Refresh	The buffer becomes a searchable segment. About a second
Flush	Segments are written to disk and the translog is cleared
Merge	Small segments are combined, deleted documents dropped
Delete	Marked deleted, and actually removed at the next merge

Refresh makes a document searchable. Flush makes it durable. They are different operations on different schedules, and the translog is what keeps a document safe between the two.

13. Field

A field is one key and value inside a document. Its type comes from the index mapping, which section 14 covers.

`text` against **`keyword`** is the distinction that matters most. A **`text`** field is broken into tokens, so it searches well and cannot be used for an exact match, a sort or an aggregation. A **`keyword`** field is stored whole, so it does all three and does not search as text.

Because both are usually wanted, the standard mapping indexes a field twice:

```
"name": {
  "type": "text",
  "fields": {
    "keyword": { "type": "keyword", "ignore_above": 256 }
  }
}
```

`name` then searches as text and `name.keyword` sorts, filters and aggregates. Dynamic mapping does this automatically for strings, which is why `somefield.keyword` appears without anyone creating it.

14. Mapping

A mapping is the schema of an index: which fields exist and what type each one is.

Type	For
<code>text</code>	Full-text search. Analyzed into tokens
<code>keyword</code>	Exact values. Not analyzed
<code>long</code> , <code>double</code>	Numbers and range queries
<code>date</code>	Dates and range queries
<code>boolean</code>	True or false
<code>object</code> , <code>nested</code>	Documents inside documents

Explicit mapping declares the fields before anything is indexed.

```
PUT /products
{
  "mappings": {
    "properties": {
      "name": { "type": "text" },
      "category": { "type": "keyword" },
      "price": { "type": "double" },
      "createdAt": { "type": "date" }
    }
  }
}
```

Dynamic mapping infers the type from the first document that contains the field.

First value seen	Becomes
"audio"	text , with a .keyword sub-field
42	long
12.5	float
true	boolean
"2026-09-16"	date , when date detection is on

A field's type cannot be changed once it exists. New fields can be added to a mapping at any time; an existing one cannot be retyped, because the inverted index was already built from it. Retyping therefore needs a new index and a reindex, for the same reason the shard count does in section 11, and the alias is the same answer.

Dynamic mapping is convenient and worth turning off in production. A number that arrives quoted once is a text field forever, and a document with unpredictable field names grows the mapping until the cluster struggles. "dynamic": "strict" rejects unknown fields instead of guessing at them.

15. Analyzers

An analyzer turns text into the tokens that go into the inverted index. It runs when a document is indexed, and again on the query.

character filters → tokenizer → token filters

Analyzer	Does	"Wireless Headphones!" becomes
Standard	Splits on word boundaries, lowercases. The default	[wireless, headphones]
Keyword	Does not split at all	[Wireless Headphones!]
Custom	Whatever the parts are configured to do	Anything

The keyword analyzer is what a keyword field effectively uses, which is why such a field matches only the whole original string.

A custom analyzer is assembled from the three stages.

```

"analysis": {
  "analyzer": {
    "product_name": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": ["lowercase", "asciifolding", "porter_stem"]
    }
  }
}

```

The same analyzer runs on the query, and that is the whole trick. Searching for "Headphones" matches a document containing "headphone" because both sides are reduced to the same token. When the index-time and search-time analyzers differ, matches fail silently and nothing reports an error.

`_analyze` shows exactly what an analyzer produces, which settles most arguments in seconds:

```

POST /_analyze
{ "analyzer": "standard", "text": "Wireless Headphones!" }

```

16. `match` against `term`

This is the most common mistake in Elasticsearch, and it has one cause.

`match` analyzes the query text. `term` does not.

```

{ "match": { "name": "Wireless Headphones" } }    finds the document
{ "term":   { "name": "Wireless Headphones" } }    finds nothing

```

The field holds the tokens `[wireless, headphones]`. `match` reduces the query to those same tokens and finds them. `term` looks for one token spelled `Wireless Headphones`, and no such token exists.

	<code>match</code>	<code>term</code>
Analyzes the query	Yes	No
Case sensitive	No, both sides are lowercased	Yes, exactly as stored
Use on	<code>text</code> fields	<code>keyword</code> , numbers, dates, booleans
Typical use	A search box	Filters, ids, status values

The rule is `match` on `text`, `term` on `keyword`. A `term` query against a text field is the usual answer to "why does my query return nothing", and a `match` query against a keyword field quietly works while doing more than it needs to.

17. Query against filter

Both select documents. The difference is whether the result is scored, and whether it can be cached.

	Query context	Filter context
Asks	How well does this match?	Does this match, yes or no?
Produces a <code>_score</code>	Yes	No
Cached	No	Yes
Speed	Slower	Faster
Suits	Relevance, from free text	Exact values, ranges, dates

```
{
  "query": {
    "bool": {
      "must": [ { "match": { "name": "wireless headphones" } } ],
      "filter": [ { "term": { "category": "audio" } },
                  { "range": { "price": { "lte": 200 } } } ]
    }
  }
}
```

`must` scores, so the text match decides the order of the results. `filter` does not, so the category and the price range only include or exclude.

Clause	Context	Effect
<code>must</code>	Query	Must match, and contributes to the score
<code>should</code>	Query	Optional, and raises the score when it matches
<code>filter</code>	Filter	Must match, contributes nothing to the score
<code>must_not</code>	Filter	Must not match

Put every yes-or-no condition in `filter`. It skips the scoring work and the result is cached for the next query that asks the same thing, which on a busy index is the single easiest performance win available.

18. Cluster architecture in depth

Section 8 showed what the pieces are. This is how the cluster decides and how it survives losing a node.

The master assigns every shard, under two rules that never bend: a replica is never placed on the node holding its primary, and shards are rebalanced when nodes join or leave. Allocation awareness extends the first rule to racks or availability zones, so a whole zone can fail without losing data.

Master election needs a majority, and that is why the count is odd.

Master-eligible nodes	Majority	Survives
1	1	No failure
2	2	No failure, and either loss stops the cluster
3	2	One failure
5	3	Two failures

Two master-eligible nodes is worse than one. Losing either leaves one node unable to form a majority, so the cluster stops rather than continuing with a split brain. Three is the normal answer.

Nodes can be tiered by how old the data is. Recent indices sit on fast disks, older ones move to cheaper storage, and lifecycle policies do the moving.

Tier	Holds
Hot	Data being written and searched constantly
Warm	Read-only, still searched often
Cold, frozen	Rarely searched, on the cheapest storage

Scaling is a question of which symptom is present:

Symptom	What to add
Searches are slow and CPU is high	Data nodes, or more replicas
Indexing throughput is low	Primary shards, on a new index, and faster disks
The master is unstable	Dedicated master nodes, and fewer shards overall
Disks are filling	Nodes, or a colder tier for old indices

Replicas help reads and cost writes. Each one is another copy every index request has to reach, so they are a search lever and never an indexing one.

19. Indexing and refresh

The write path has a fixed order, and each stage is a place to tune:

```
request → primary shard → translog and buffer → replicas → acknowledged
      → refresh (searchable) → flush (durable) → merge (space reclaimed)
```

Bulk requests are the largest single lever. One request carrying many documents removes the per-request overhead entirely.

```
POST /_bulk
{"index":{"_index":"products","_id":"41"}}
{"name":"Wireless Headphones","price":129.99}
{"index":{"_index":"products","_id":"42"}}
{"name":"USB-C Cable","price":9.99}
```

Somewhere between five and fifteen megabytes per request is a reasonable target. Larger stops helping and starts causing rejections.

A bulk load should turn off the things it does not need.

```
PUT /products/_settings
{ "index": { "refresh_interval": "-1", "number_of_replicas": 0 } }
```

Setting during a load	Why
<code>refresh_interval: -1</code>	No segment per second, so far less merging afterwards
<code>number_of_replicas: 0</code>	The work is done once rather than twice
No explicit <code>_id</code>	Skips the check for an existing document

Both settings are restored when the load finishes, and the replicas are then built once from a settled index rather than continuously during the write.

The translog is what makes an unflushed write safe. Every write is appended to it before the request is acknowledged, so a crash replays anything not yet flushed. By default that is an fsync per request, which is safe and not free; `async` durability is faster and can lose a few seconds of writes.

20. Performance optimization

Most Elasticsearch performance work is one of the items below, and they are in rough order of how often they are the answer.

On the search side:

Do this	Because
Put non-scored conditions in <code>filter</code>	Free to repeat once cached. Section 17
Use <code>search_after</code> , never deep <code>from</code>	See below. This is the big one
Return only the fields needed	Less to read, merge and send
Avoid leading wildcards and regex	They cannot use the inverted index
Avoid scripts in queries	They run per document, per shard

Deep pagination is the classic way to bring down a cluster. A request for `from: 10000, size: 10` makes every shard collect 10,010 hits and the coordinating node sort all of them, to return ten. `search_after` carries the sort values of the last result instead, so each page costs the same as the first.

On the mapping side:

Do this	Because
Declare the mapping explicitly	Dynamic mapping guesses, and guesses stick
Set <code>"index": false</code> on fields only ever returned	They are stored but not indexed
Keep shards in the tens of gigabytes	Too many small shards is pure overhead
Force-merge indices that are now read-only	One segment searches faster than fifty

On the cluster side:

Setting	Value
JVM heap	Half the available memory
Heap ceiling	Around 31 GB, never more
The other half of memory	Left free, deliberately
Disk	Local SSD

The free memory is not waste, it is the most important setting here. Lucene reads segments through the operating system's file cache, so the memory left outside the heap is what keeps searches off the disk. A machine with all its RAM given to the JVM is slower than one with half.

Heap above about 31 GB is slower than heap below it. Past that threshold the JVM can no longer compress object pointers, so every reference doubles in size and less fits in cache. Two nodes with 31 GB beat one with 64 GB.

Measure rather than guess. The profile API breaks a query into its parts and times each one, and the slow log records the queries actually worth looking at.

```
POST /products/_search
{ "profile": true, "query": { "match": { "name": "headphones" } } }
```

21. Exceptions and failure handling

Exception	Status	Means	Do
<code>version_conflict_engine_exception</code>	409	Two writes to one document at once	Retry, or set <code>retry_on_conflict</code>

Exception	Status	Means	Do
<code>mapper_parsing_exception</code>	400	A value does not fit its mapped type	Fix the document, or the mapping
<code>index_not_found_exception</code>	404	No such index or alias	Create it, or check the alias
<code>circuit_breaking_exception</code>	429	The request would use too much heap	Ask for less. Fewer buckets, smaller pages
<code>es_rejected_execution_exception</code>	429	The thread pool queue is full	Back off and retry
<code>cluster_block_exception</code>	403	The index is read-only	Usually disk. See below

429 means back off, not fail. A rejection is Elasticsearch protecting itself from a request it cannot serve right now. A client that treats it as a permanent error throws away work that a retry a second later would have completed.

A search can partly succeed and still return 200. Every response carries a `_shards` section, and a failed shard there means the results are incomplete.

```
"_shards": { "total": 5, "successful": 4, "failed": 1 }
```

Code that reads `hits` without checking `_shards` silently returns partial answers, and nothing in the status code says so.

A bulk request is the same trap, one level down. The request returns 200 even when individual documents failed. The response has `"errors": true` and a status per item, and that is the only place the failure appears.

Disk watermarks are the usual cause of a read-only index.

Watermark	Default	Effect
Low	85%	No new shards are placed on the node
High	90%	Shards are moved off it
Flood stage	95%	Every index on it becomes read-only

The flood stage block does not clear itself. Freeing disk is necessary and not sufficient; the read-only block stays until it is removed explicitly, which is why an index can stay unwritable long after the disk problem is fixed.

```
PUT /products/_settings
{ "index.blocks.read_only_allow_delete": null }
```

The pieces together

Piece	What it is
Cluster	The whole system, under one name
Node	One process, and the roles it is given
Index	Where documents of one kind live
Shard	How an index is divided and spread out
Replica	Why losing a node loses nothing
Document	One JSON object with an id
Field	One key and value, with a mapped type
Inverted index	Why searching is a lookup and not a scan
<code>text</code> and <code>keyword</code>	Searchable against exact
Mapping	The schema, and why a type cannot be changed
Analyzer	What turns text into tokens, on both sides
<code>match</code> and <code>term</code>	Analyzed against literal
Filter context	Unscored, cached, and faster
Quorum	Why master-eligible nodes come in threes
Bulk and <code>refresh_interval</code>	The two levers that matter when loading
<code>search_after</code>	Why deep pagination is never the answer
Free memory	What the file cache uses, and why heap stops at 31 GB
<code>_shards</code> and bulk <code>errors</code>	Where a partial failure actually appears
Near real time	Indexed now, searchable in about a second

For example, a product catalog. Products live in PostgreSQL, which owns them, and every change is copied into an Elasticsearch index with two shards and one replica each. A shopper searches for "wireless headphones"; the request reaches any node, which asks one copy of each shard, merges the scored results and returns the best ten in a few milliseconds. A product added a moment ago may not appear until the next refresh. The category filter and the sort use `category.keyword` rather than `category`, because the analyzed field cannot do either.

The sentence that explains most of the design is that Elasticsearch does its work at write time. Analysis, tokenizing and building the inverted index all happen when the document is indexed, which is why searching is fast and why a document is not searchable the instant it arrives.